Shift Registers

Digital System Fundamentals

Sorayut Glomglome

Learning Outcome

- Identify the basic forms of data movement in shift registers.
- Explain how serial in/serial out, serial in/parallel out, parallel in/serial out, and parallel in/parallel out shift registers operate.
- Describe how a bidirectional shift register operates
- Use a shift register as a time-delay device
- Use a shift register to implement a serial-to-parallel data converter

CONTENTS

1. Shift Registers

2. Shift Register Applications

1. Shift Registers

A Register

- A ***register*** is a digital circuit with two basic functions: *data storage* and *data movement*.
- The storage capability of a register makes it an important type of *memory device*.

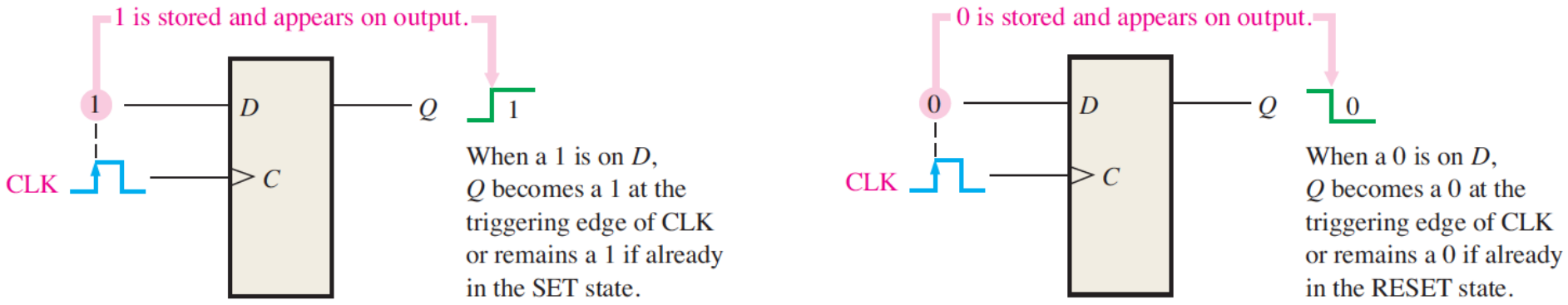


FIGURE 8–1 The flip-flop as a storage element.

A Register

- The *storage capacity* of a register is the total number of bits (1s and 0s) of digital data it can retain.
- Each stage (flip-flop) in a register represents one bit of storage capacity.
- The number of stages in a register determines its storage capacity.
- The *shift capability* of a register permits the movement of data from stage to stage *within the register* or *into or out of the register* upon application of clock pulses.

Shift Registers

- Shift registers consist of arrangements of flip-flops and are important in applications involving the *storage* and *transfer* of data in a digital system.

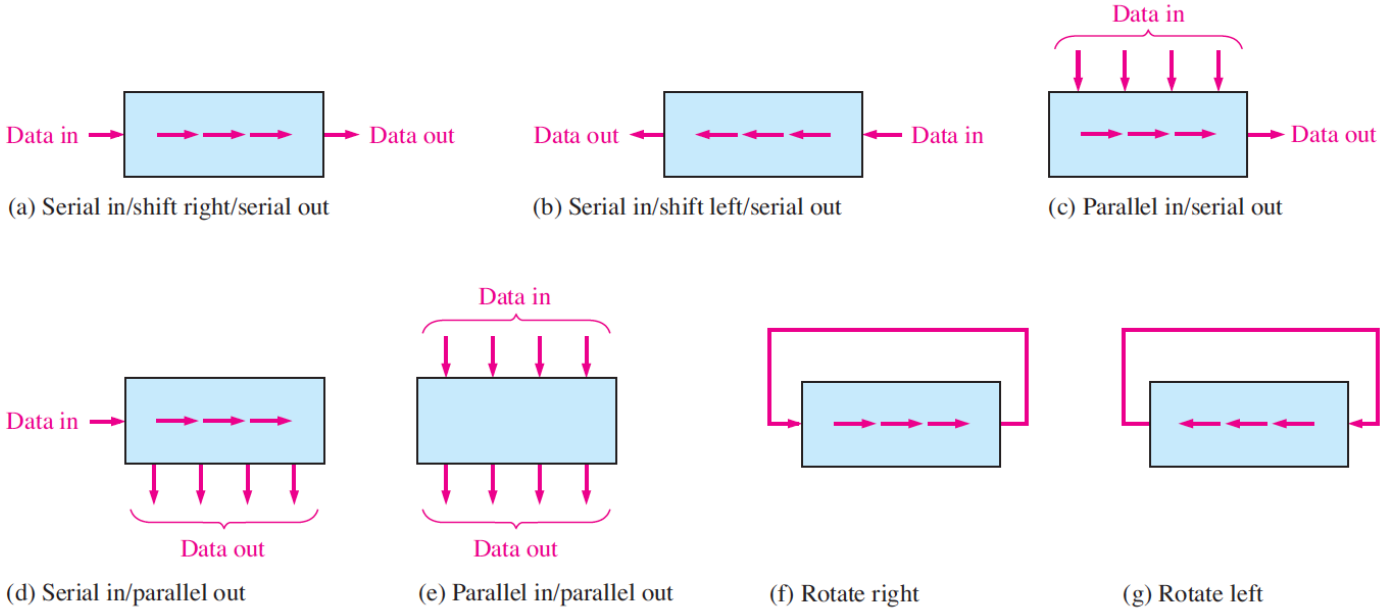


FIGURE 8–2 Basic data movement in shift registers. (Four bits are used for illustration. The bits move in the direction of the arrows.)

Serial In/Serial Out Shift Registers

- The serial in/serial out shift register accepts data serially—that is, *one bit at a time* on a single line.
- It produces the stored information on its output also in serial form.
- Figure 8–3 shows a 4-bit device implemented with D flip-flops. With four stages, this register can store up to four bits of data.

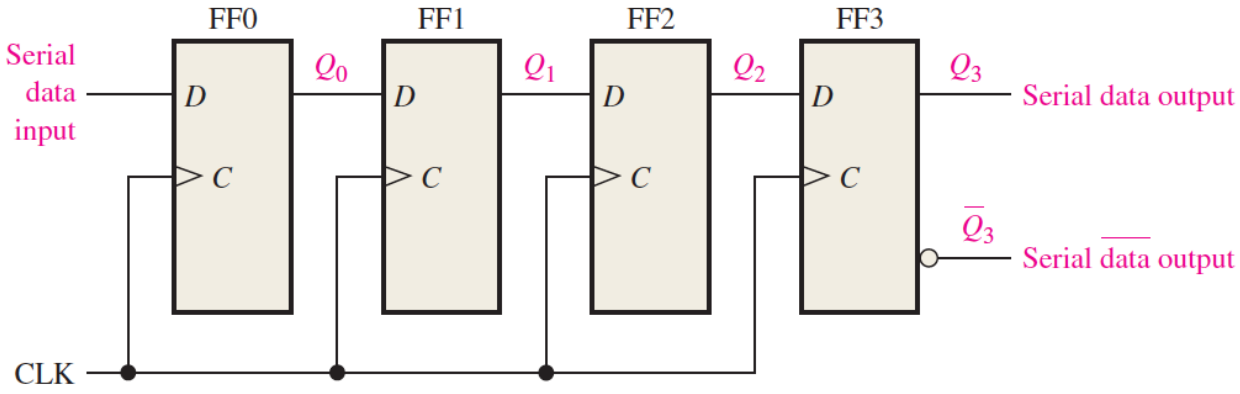


FIGURE 8–3 Serial in/serial out shift register.

1 0 1 0

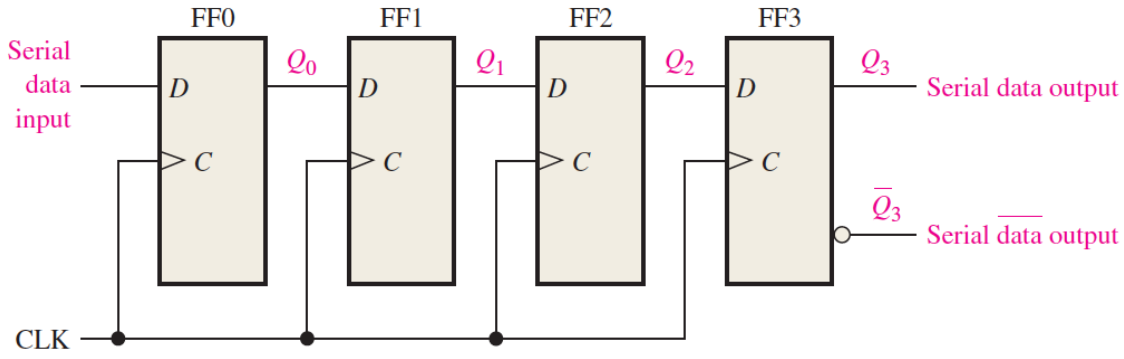


TABLE 8–1

Shifting a 4-bit code into the shift register in Figure 8–3. Data bits are indicated by a beige screen.

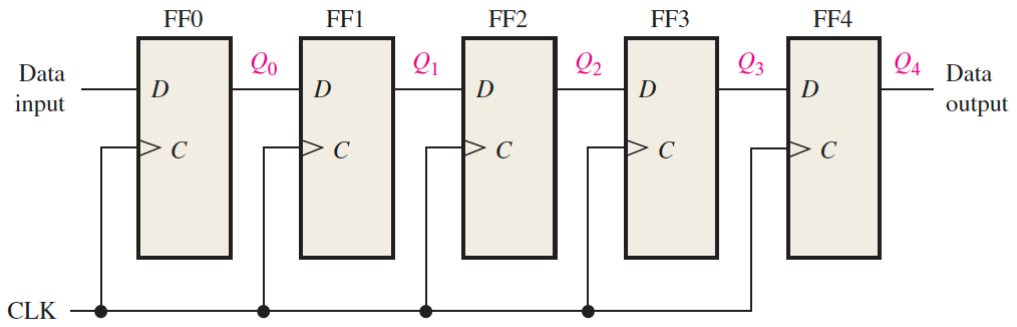
CLK	FF0 (Q_0)	FF1 (Q_1)	FF2 (Q_2)	FF3 (Q_3)
Initial	0	0	0	0
1	**0**	0	0	0
2	**1**	**0**	0	0
3	**0**	**1**	**0**	0
4	**1**	**0**	**1**	**0**

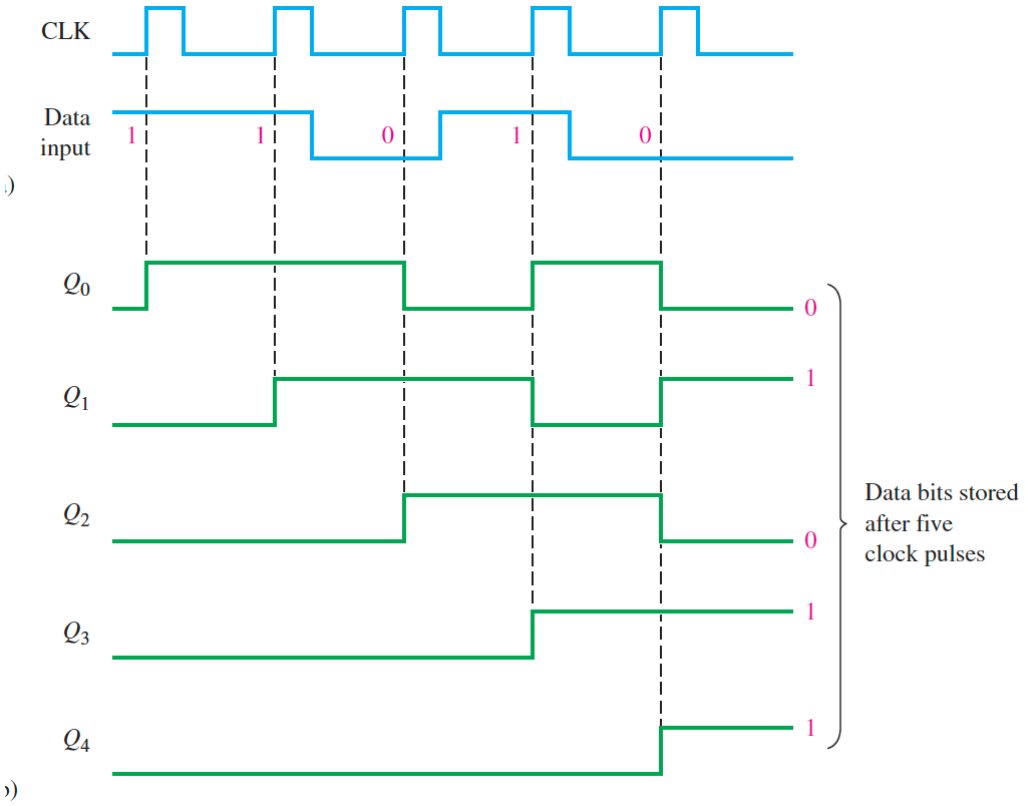
TABLE 8–2

Shifting a 4-bit code out of the shift register in Figure 8–3. Data bits are indicated by a beige screen.

CLK	FF0 (Q_0)	FF1 (Q_1)	FF2 (Q_2)	FF3 (Q_3)
Initial	**1**	**0**	**1**	**0**
5	0	**1**	**0**	**1**
6	0	0	**1**	**0**
7	0	0	0	**1**
8	0	0	0	0

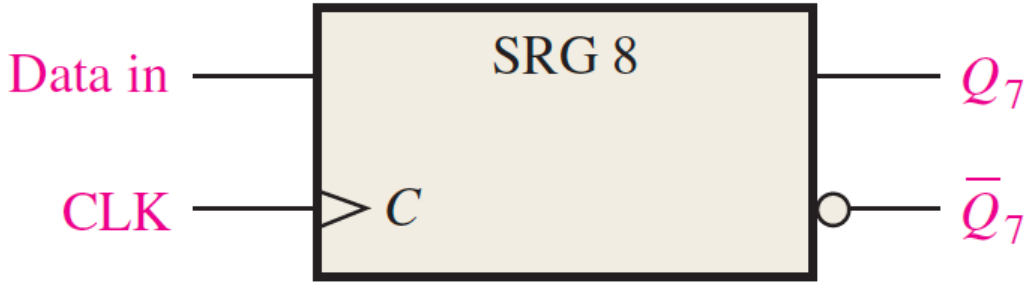
1 1 0 1 0


FF0
FF1
FF2
FF3
FF4
Data input
D
C
Q_0
Q_1
Q_2
Q_3
Q_4
Data output
CLK


CLK
Data input
1
1
0
1
0
Q_0
Q_1
Q_2
Q_3
Q_4
0
1
0
1
1
Data bits stored after five clock pulses

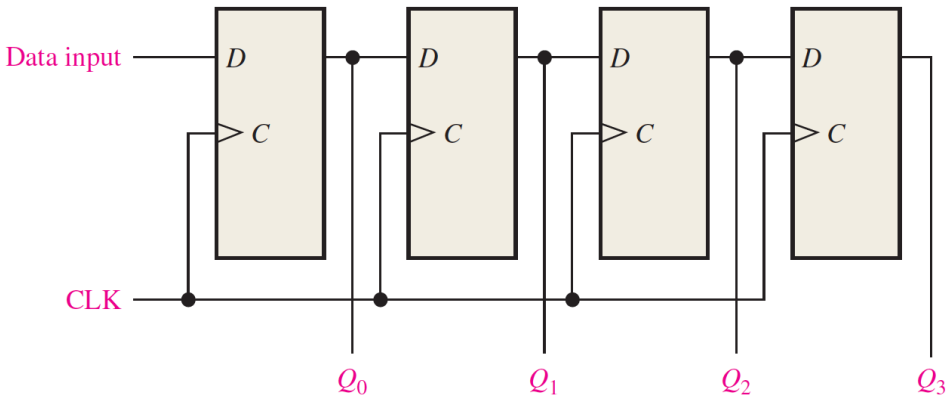
Logic symbol for an 8-bit serial in/serial out shift register.

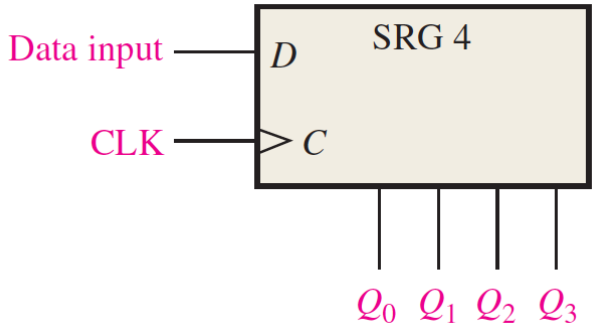
- The “SRG 8” designation indicates a shift register (SRG) with an 8-bit capacity.



Serial In/Parallel Out Shift Registers

- Data bits are entered serially (least-significant bit first) into a serial in/parallel out shift register.
- The data bits are taken out of the register in the parallel output register.
- All bits are available simultaneously, rather than on a bit-by-bit basis





Show the states of the 4-bit register (SRG 4) for the data input and clock waveforms in Figure 8–7(a). The register initially contains all 1s.

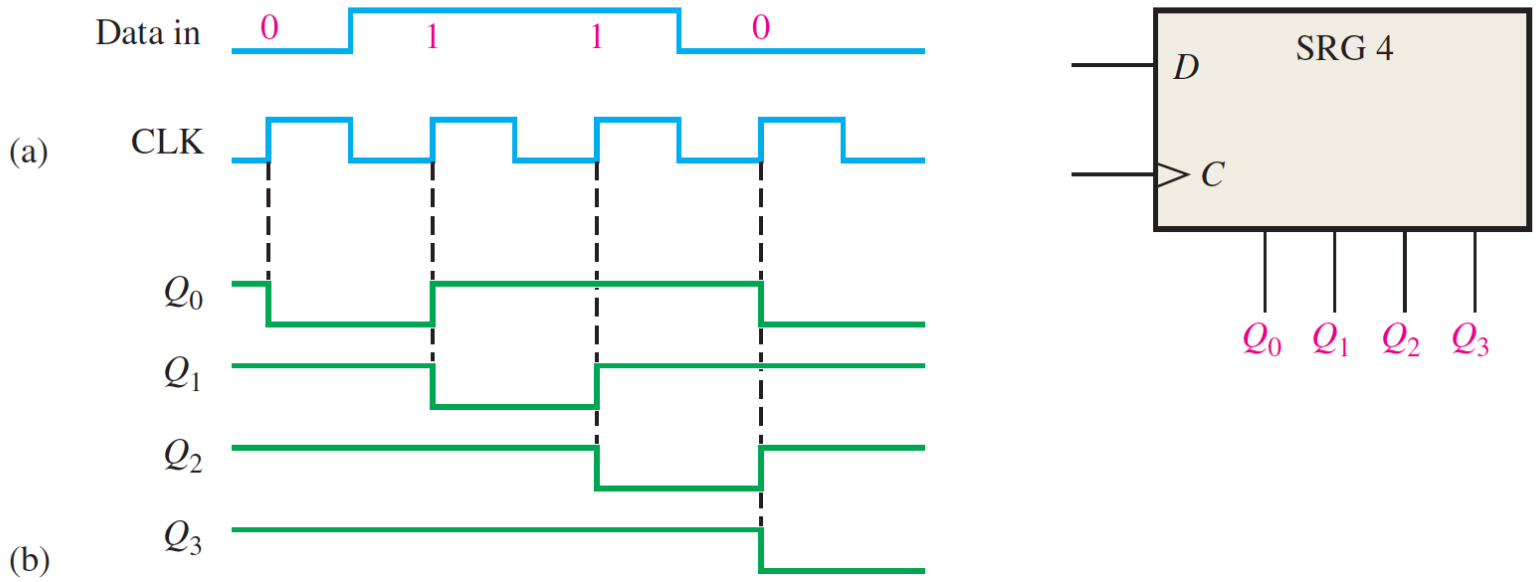
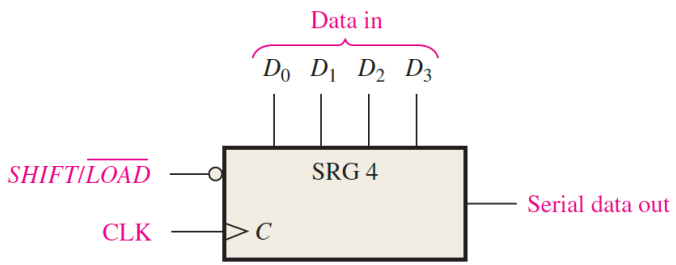


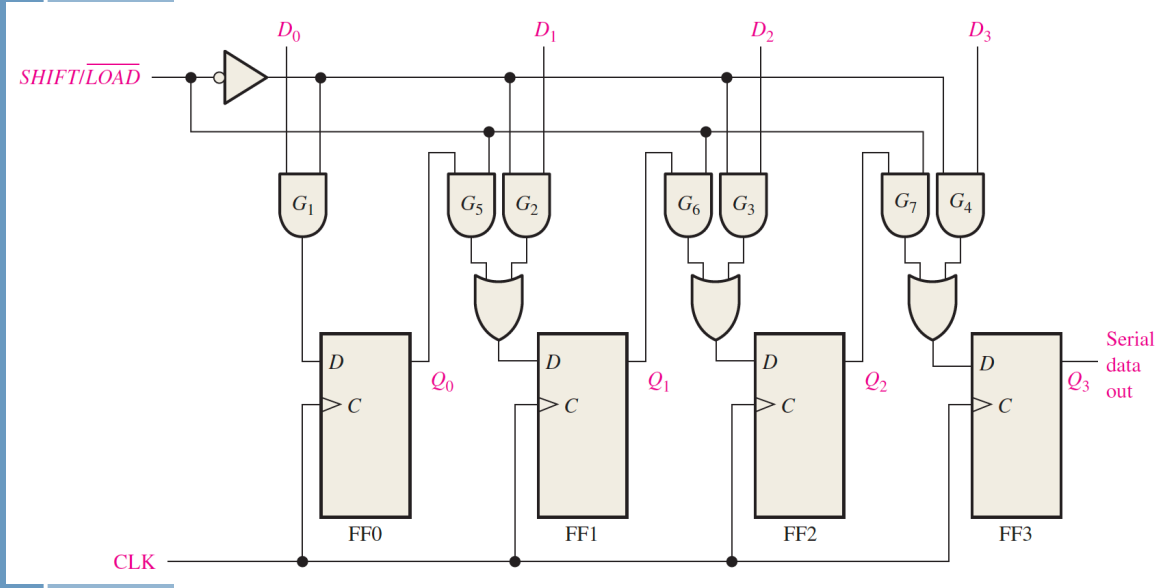
FIGURE 8–7

Solution

The register contains 0110 after four clock pulses. See Figure 8–7(b).

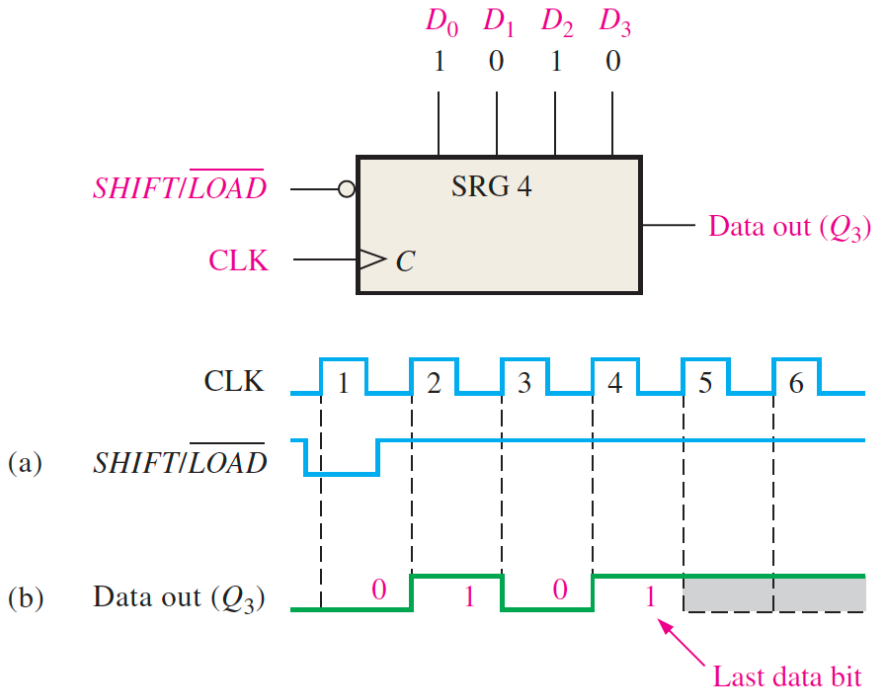
Parallel In/Serial Out Shift Registers





- When SHIFT/LOAD is LOW, gates G1 through G4 are enabled, allowing each data bit to be applied to the D inputs of flip-flops.
- When a clock pulse is applied, the flip-flops with D = 1 will set and those with D = 0 will reset.
- When SHIFT/LOAD is HIGH, gates G1 through G4 are disabled and gates G5 through G7 are enabled, allowing the data bits to shift right from one stage to the next.
- The OR gates allow either the normal shifting operation or the parallel data-entry operation,

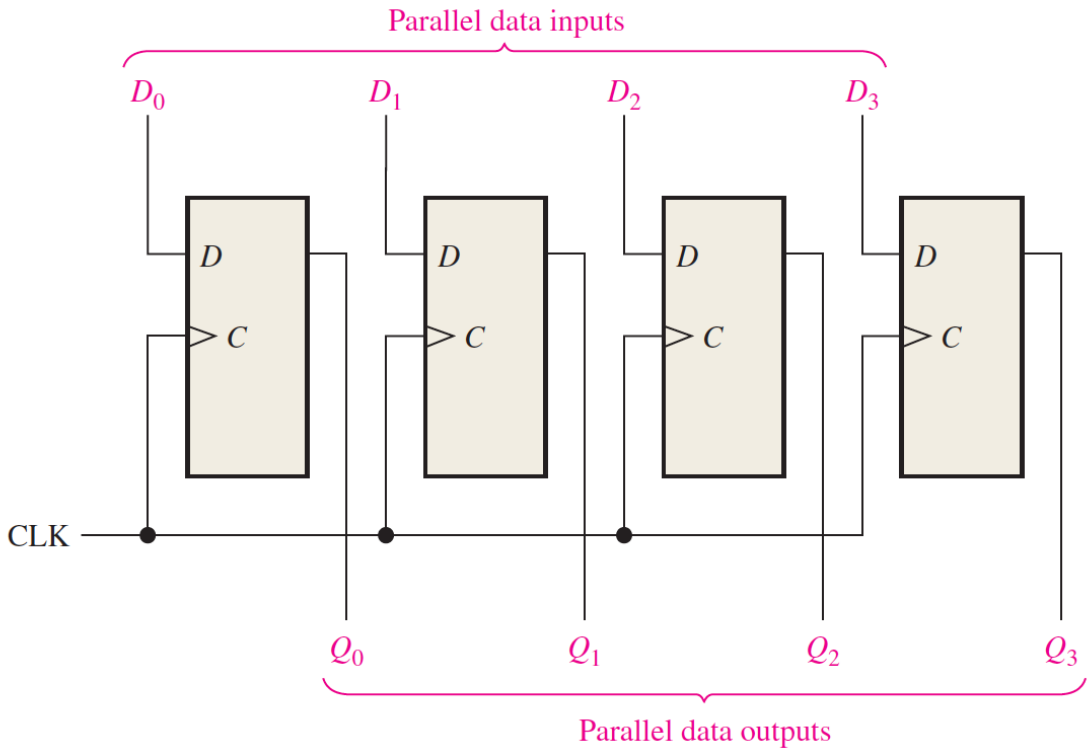
Show the data-output waveform for a 4-bit register with the parallel input data and the clock and $SHIFT/\overline{LOAD}$ waveforms given in Figure 8–11(a). Refer to Figure 8–10(a) for the logic diagram.



Solution

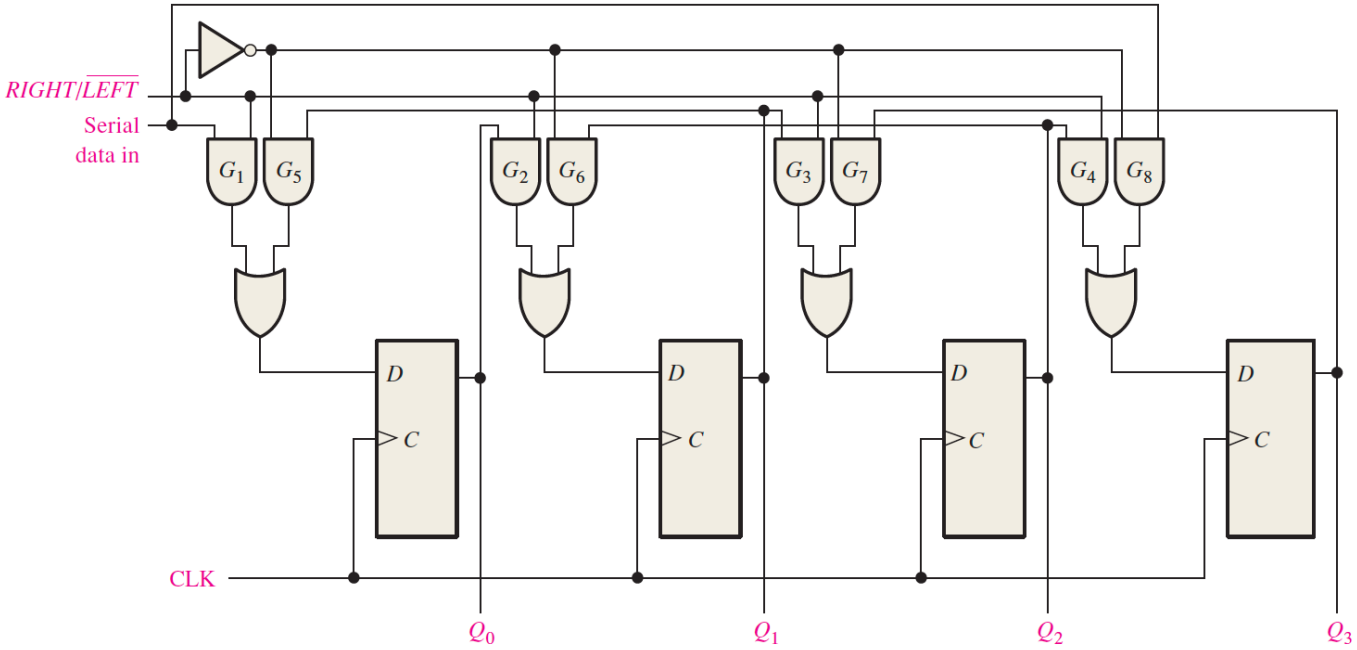
On clock pulse 1, the parallel data ($D_0D_1D_2D_3 = 1010$) are loaded into the register, making Q_3 a 0. On clock pulse 2 the 1 from Q_2 is shifted onto Q_3; on clock pulse 3 the 0 is shifted onto Q_3; on clock pulse 4 the last data bit (1) is shifted onto Q_3; and on clock pulse 5, all data bits have been shifted out, and only 1s remain in the register (assuming the D_0 input remains a 1). See Figure 8–11(b).

Parallel In/Parallel Out Shift Registers

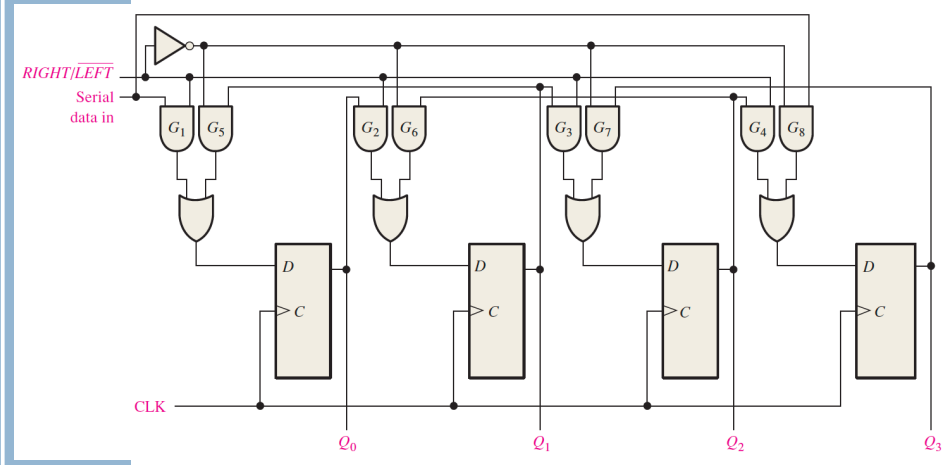


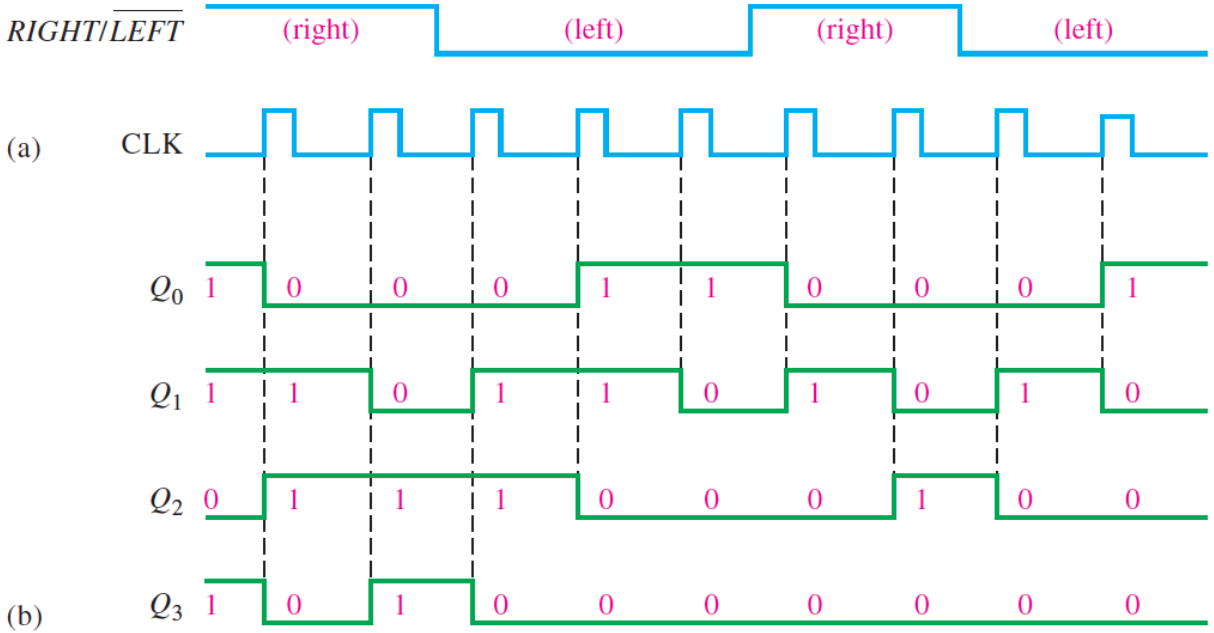
Bidirectional Shift Registers

- A *bidirectional* shift register is one in which the data can be shifted either left or right.
- It can be implemented by using gating logic that enables the transfer of a data bit from one stage to the next stage to the right or to the left, depending on the level of a control line.



Determine the state of the shift register of Figure 8–17 after each clock pulse for the given $RIGHT/\overline{LEFT}$ control input waveform in Figure 8–18(a). Assume that $Q_0 = 1$, $Q_1 = 1$, $Q_2 = 0$, and $Q_3 = 1$ and that the serial data-input line is LOW.

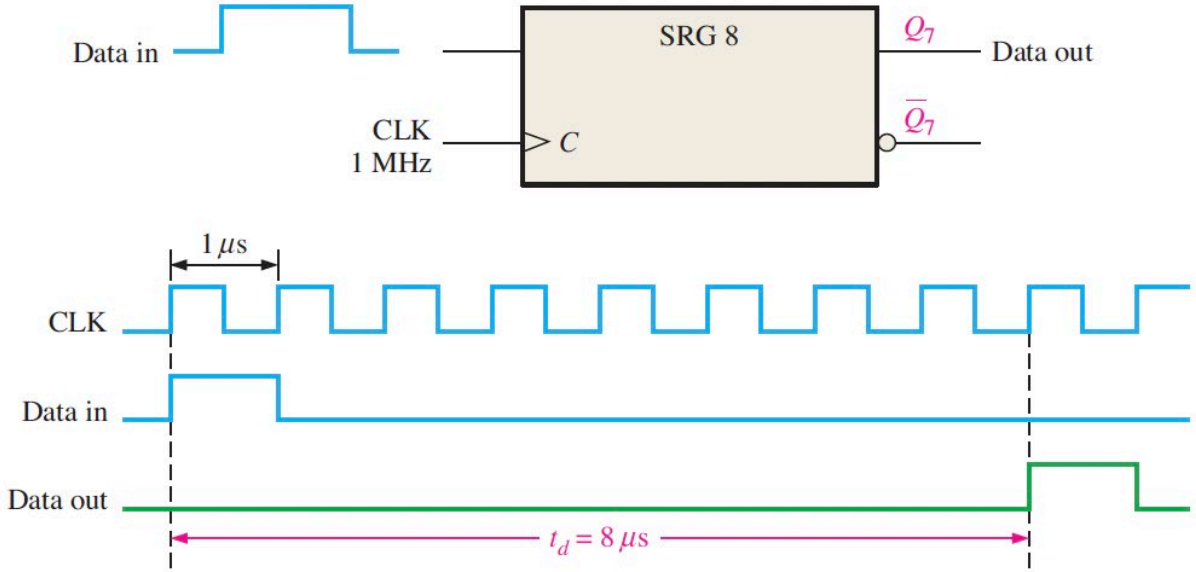




2. Shift Register Applications

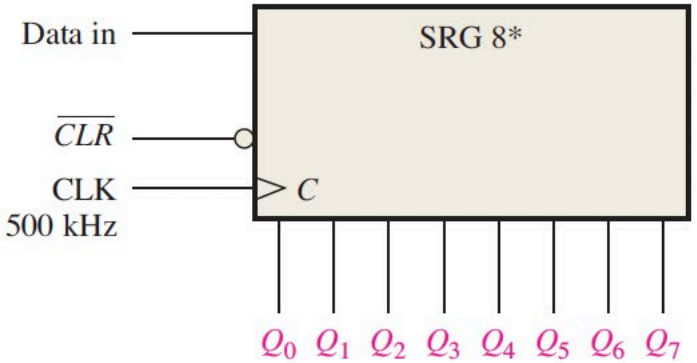
Time Delay

- A serial in/serial out shift register can be used to provide a time delay from input to output that is a function of both the number of stages (n) in the register and the clock frequency.



Example

Determine the amount of time delay between the serial input and each output in Figure 8–27. Show a timing diagram to illustrate.



* Data shifts from Q_0 toward Q_7.

FIGURE 8–27

Example

Solution

The clock period is 2 μs. Thus, the time delay can be increased or decreased in 2 μs increments from a minimum of 2 μs to a maximum of 16 μs, as illustrated in Figure 8–28.

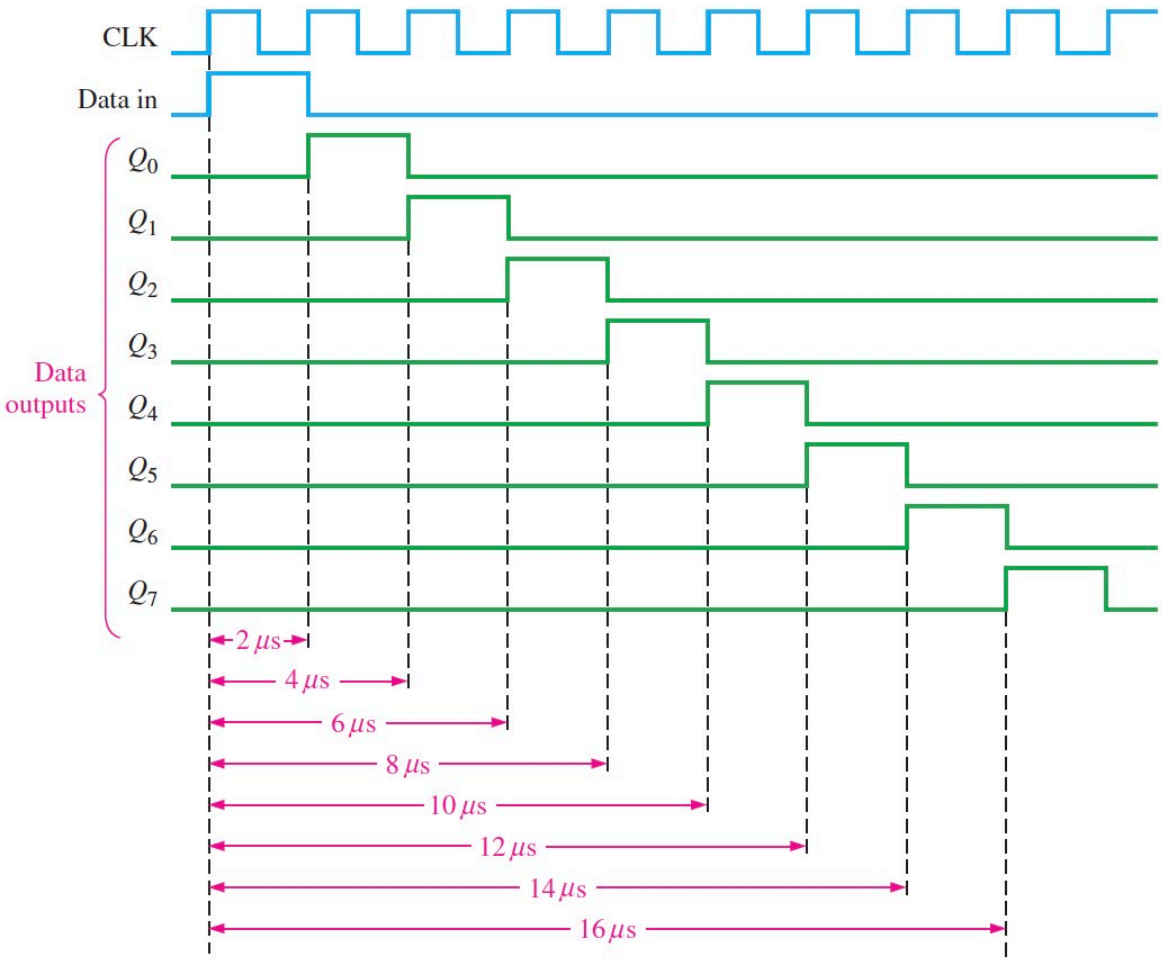
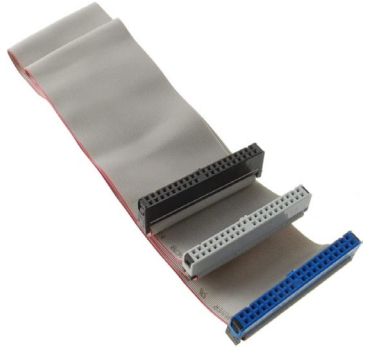


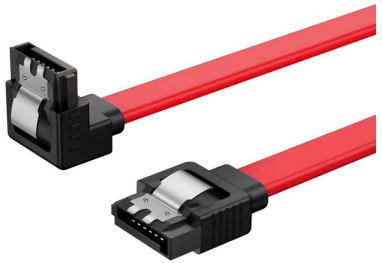
FIGURE 8–28 Timing diagram showing time delays for the register in Figure 8–27.

Serial-to-Parallel Data Converter

- Serial data transmission from one digital system to another is commonly used to reduce the number of wires in the transmission line.
- For example, eight bits can be sent serially over one wire, but it takes eight wires to send the same data in parallel.
- Serial data transmission is widely used by peripherals to pass data back and forth to a computer.
- For example, USB (universal serial bus) is used to connect keyboards printers, scanners, and more to the computer.
- All computers process data in parallel form, thus the requirement for serial-to-parallel conversion.

PATA vs SATA





PATA vs SATA

S.NO	PATA	SATA
1.	PATA stands for Parallel Advanced Technology Attachment.	SATA stands for Serial Advanced Technology Attachment.
2.	It is a 40 pin connector.	It is a 7 pin connector.
3.	It is high in cost.	It is cheaper in cost.
4.	The speed of data transfer is lower.	The speed of data transfer is higher.
5.	Power consumption is more.	Power consumption is less.
6.	The cable size is bigger.	The cable size is smaller.
7.	It doesn't have the feature of hot swapping.	It has the feature of hot swapping.
8.	External hard drives cannot be used.	External hard drives can be used.

PATA stands for **Parallel Advanced Technology Attachment** which is a bus interface used for connecting secondary storage devices like hard disks, optical drives. It was first introduced in the year 1986 by Western Digital and Compaq. It was later replaced by SATA.

SATA stands for **Serial Advanced Technology Attachment** is a bus interface that connects hard disks, optical drives. It was introduced in 2001 after **PATA** was slowly declining its demand by Serial ATA Working Group. SATA has more advantages than PATA making its demand more.

Difference Between PATA and SATA - GeeksforGeeks

Serial-to-Parallel Data Converter

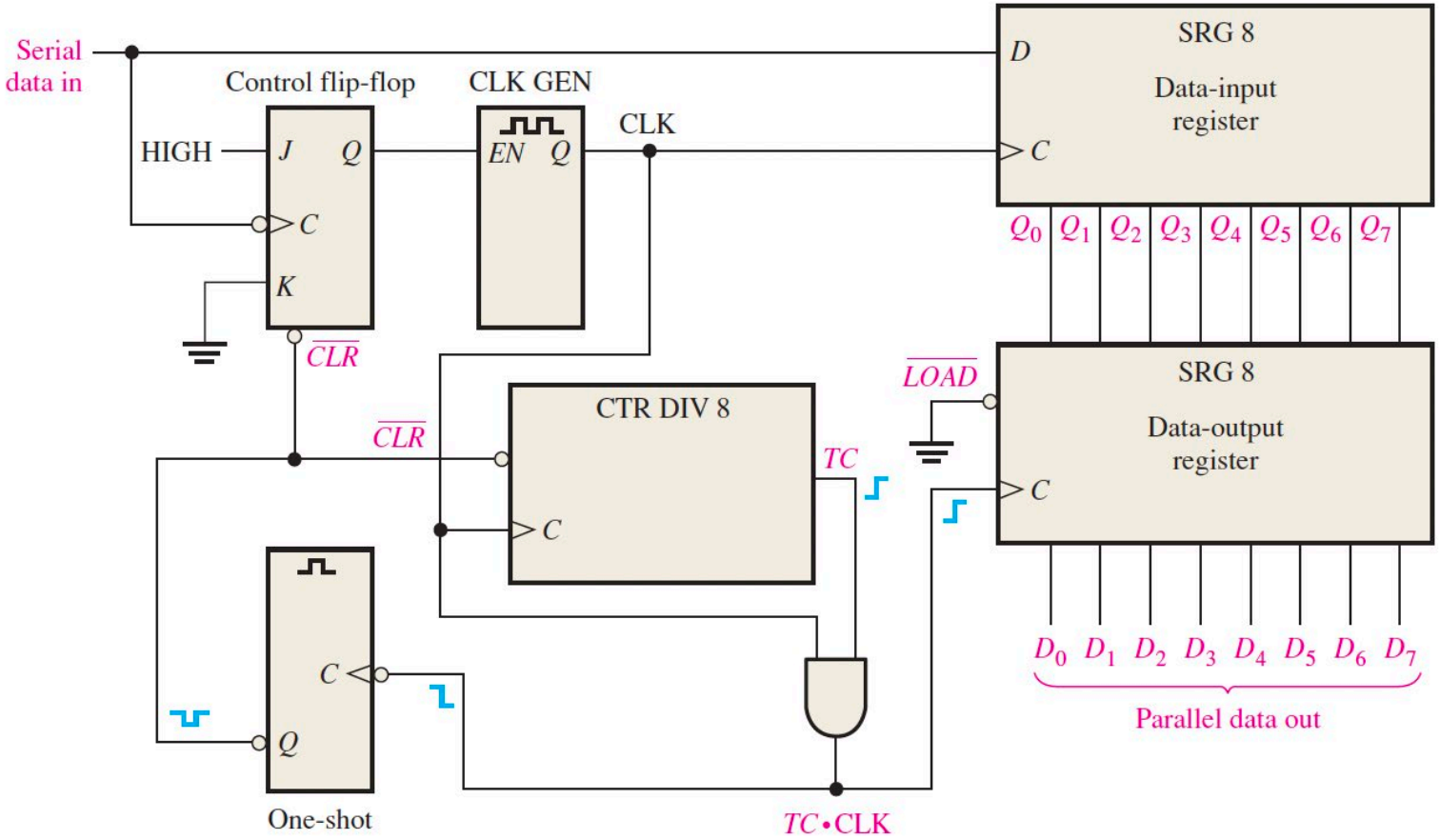


FIGURE 8–31 Simplified logic diagram of a serial-to-parallel converter.

Serial-to-Parallel Data Converter

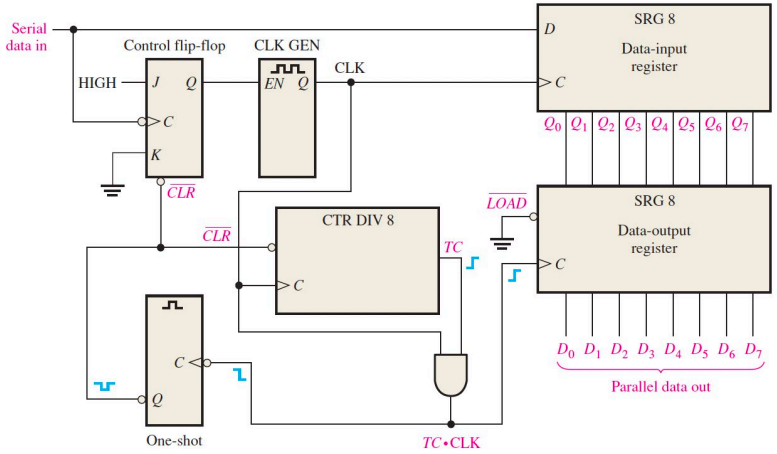


FIGURE 8–31 Simplified logic diagram of a serial-to-parallel converter.

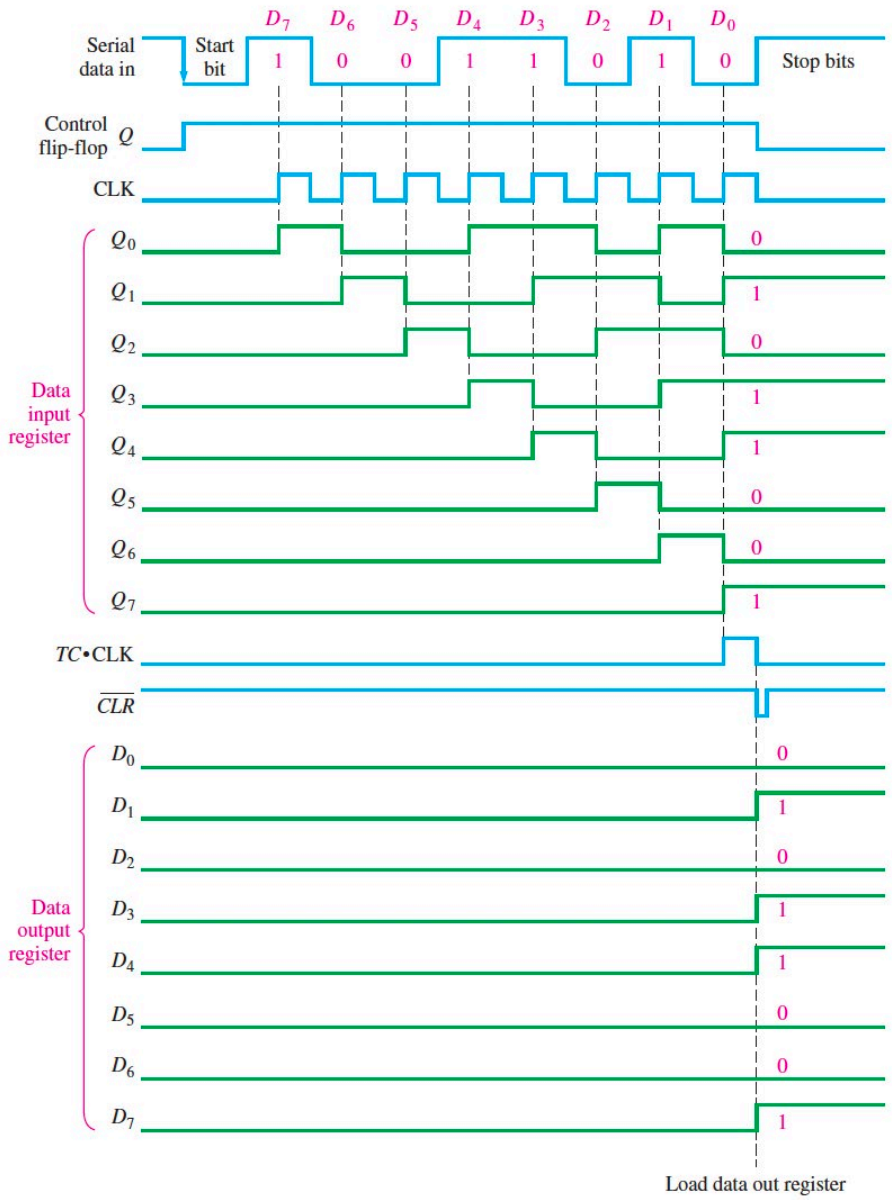


FIGURE 8–33 Timing diagram illustrating the operation of the serial-to-parallel data converter in Figure 8–31.

Universal Asynchronous Receiver Transmitter (UART)

- Computers and microprocessor-based systems often send and receive data in a parallel format.
- These systems must communicate with external devices that send and/or receive serial data.
- An interfacing device used to accomplish these conversions is the UART.

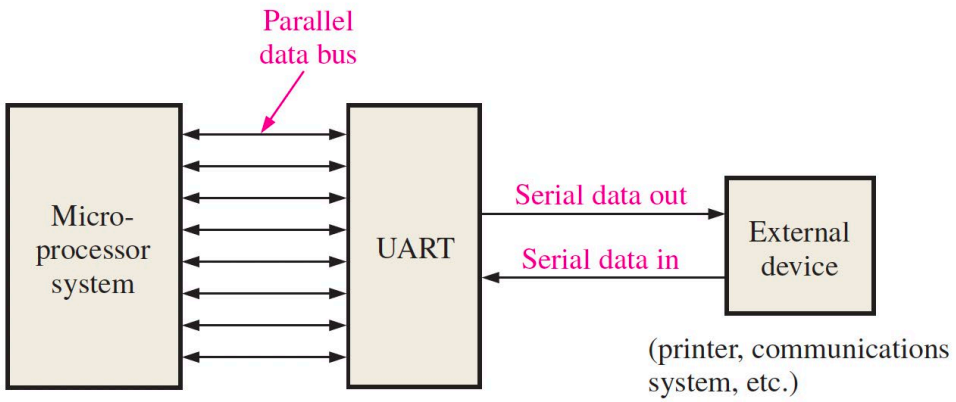
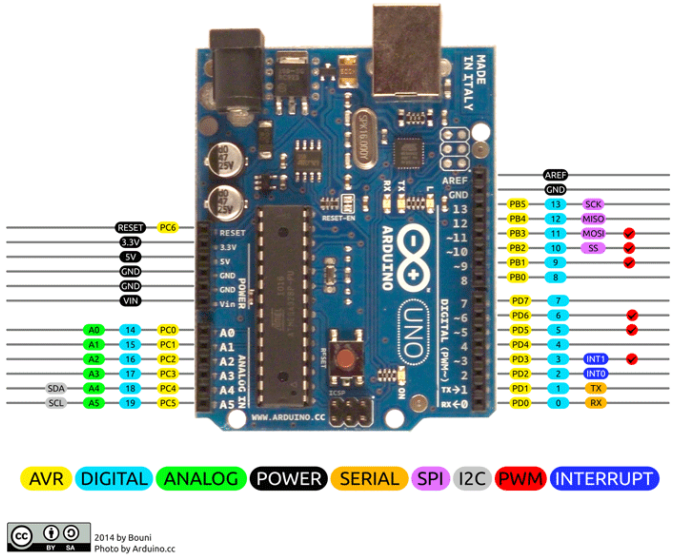


FIGURE 8–34 UART interface.



UART : Bloack Diagram

- The UART receives data in serial format, converts the data to parallel format, and places them on the data bus.
- The UART also accepts parallel data from the data bus, converts the data to serial format, and transmits them to an external device.

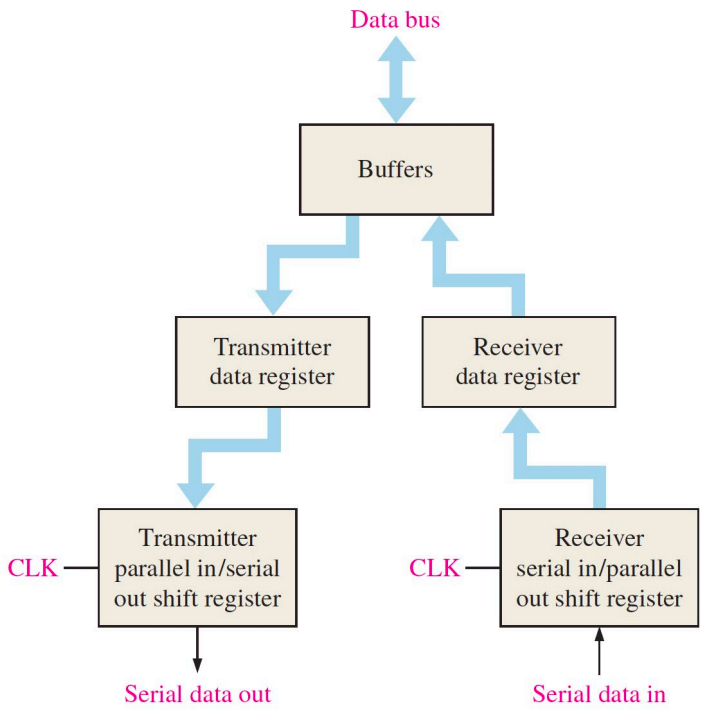


FIGURE 8–35 Basic UART block diagram.

Keyboard Encoder

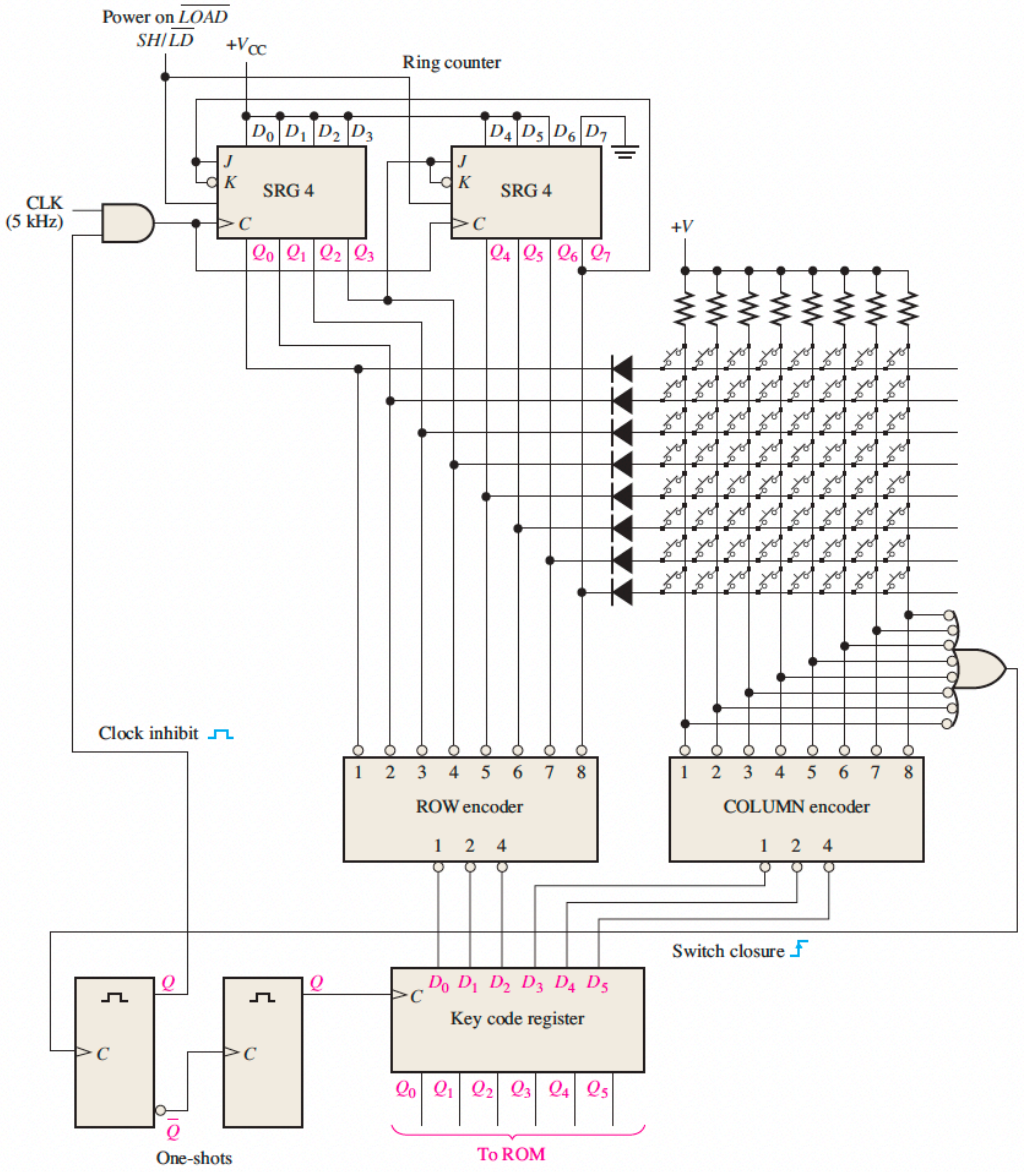


FIGURE 8–36 Simplified keyboard encoding circuit.

THANK YOU